MEETING OR CONTACT REPORT

Date of Report: February 24, 1958

Organization & Location:	Date: February 19-20, 1958
Los Alamos Scientific Laboratories Los Alamos, New Mexico	Reported By: E. F. Codd
Project: Stretch Mathematical Planning Committee Meeting #12	Department: 749
	Follow-up Date:

PERSONNEL PARTICIPATING:
(Place asterisk next to those on distribution list. Other distribution show at end of report)

	LASL	IBM
Messrs:	B. Carlson	G. A. Blaauw *
	G. L. Carter	F. P. Brooks *
	R. M. Frank	J. Cocke *
	R. B. Lazarus	W. P. Heising*
	E. L. Voorhees	F. E. Johnston*
	M. B. Wells	H. G. Kolsky*
	D. F. Woods	E. F. Codd*

1. Summary of Meeting

The main items discussed were:

1. Recent changes in the instruction set, interrupt system and special registers.
2. LASL questions concerning floating point: specifically floating divide, the noisy mode, rounding and treatment of zero.
3. LASL questions concerning the interrupt system.
4. Error detection and correction.
5. Programming Systems.
6. Console

Each of these points is treated in the correspondingly numbered appendix.

In addition, the following items were brought up but did not result in discussion:

1. I/O progress - IBM reported that the Exchange was now being constructed and development work was proceeding on tapes faster than the 729 III.

2. Test Problem - LASL indicated that preparation of this problem had not been started yet.

3. Space requirements - LASL requested information from IBM as soon as possible because of inevitable delays in getting approval for new buildings or changes in existing ones.

4. Flow charts - IBM loaned some flow charts of the computer to LASL.

5. Look ahead - IBM presented an introductory description of the scheme upon which current plans are based.

2. Future Action

It was agreed that the next meeting should be held early in May and that it would be advantageous to make it a prelude or postlude to the WJCC, May 6-8 in Los Angeles.

E. F. Codd

EFC/jcv
cc: Messrs: W. Buchholz
E. W. Coffin
S. W. Dunwell
J. C. Gibson
P. S. Herwitz
W. J. Lawless
Miss E. McDonough
Messrs: B. Moncreiff
D. W. Pendery
P. H. Sterbenz
D. W. Sweeney

Brownline to: Mr. B. Carlson
cc: Mr. Paul Wilson, Contract Officer

Appendix 1: Recent Changes

Since the last LASL-IBM meeting modifications have been made to certain operations, indicators, and special registers. All of the changes pertinent to Chapter 5 of the manual are reflected in the new edition of this chapter (issued February 12, 1958). Some changes, however, are partially or completely beyond the scope of this chapter and are, therefore, detailed below:

1. The instruction LOAD WITH FLAG is introduced in integer and single precision floating point arithmetic. These flag bits are placed in the accumulator sign register in addition to the normal LOAD functions.

2. The byte size field in the CONNECT format expresses a byte size which applies to the memory field, exactly as in decimal arithmetic. This makes these two formats identical except for the operation code and modifier field.

3. The entire register designation function for all CONNECT operations has been taken out. All CONNECT operations now use the accumulator as one operand. The field which formerly specified register designation now specifies byte size.

4. The byte size of the accumulator is no longer variable. There is no accumulator byte size register, and successive byte addresses in the accumulator are internally computed by counting rather than by addition. For decimal arithmetic, the effective accumulator byte size is four bits (counting in the third position from the right of the bit address). For CONNECT operations, the effective accumulator byte size is eight bits (counting in the fourth position from the right of the bit address).

5. The accumulator sign byte register is reduced from eight to four bits.

6. The RM byte size register is eliminated. The D data flow sequence as used in multiplication and division always proceeds with four-bit bytes in decimal, eight bit bytes in binary. The byte addresses in the D sequence can now be internally computed by counting rather than adding.

7. The operation Branch on Bit is introduced and the operation Branch on register B is deleted.

8. The register designation for Connect operations is restricted to the Accumulator and its Sign register. Connect to Mask and Indicator register are deleted.

9. All full length instructions have the code 1000 in bit positions 24-27. These instructions include Store Instruction Counter and Branch on Bit.

10. The I field, when present in bits 28-31 or 60-63, has been standardized in function, so that it is only used for address modification, including progressive indexing. The formats for the Store Address and Rename instructions are changed accordingly.

11. The identification of Branch instructions is reduced from three to two code classes. Any half-word Branch instruction may occur as the second half of a full length instruction. It is interpreted either as a Store Instruction Counter and Branch, or as a Branch on Bit operation.

12. The Name operation has been deleted and the Rename operation has been redefined. The name of the index is always stored in the Refill field of X0, storage location 16. Bit 27 of the index word in J conditions the store part of the Rename operation as will be explained in greater detail in a further communication.

13. The operations Geometric Refill and Conditional Geometric Refill have been simplified to Refill and Conditional Refill. The effective address designates the word to be refilled.

14. Geometric indexing has been reduced by restricting it to the first 16 bits of the instruction address.

15. A revised list of indicators is given on the next three pages. Note that indicators 0 through 20 are always masked ON and 50 through 63 are always masked OFF. Only indicators 21 through 49 are individually maskable and, of these, 43 through 49 will not appear in the Basic or Sigma systems.

LIST OF INDICATORS

	Mask			Reset By
		EQUIPMENT CHECK		
0	1	Control Check	*	
1	1	Arithmetic Unit Check	*	
2	1	Memory Check	*	
3	1	Exchange Check		
4	1	Incomplete Operation	*	
		ATTENTION REQUEST		
5	1	Elapsed Time		
6	1	Other CPU		
		INPUT-OUTPUT		
7	1	Unit Not Ready Reject	T	Next rejectable I/O Operation
8	1	Exchange Busy Reject	T	
9	1	Unit Busy Reject	T	
10	1	Interrupt Waiting Reject	T	
11	1	Cancel Status		
12	1	Unit Check		
13	1	End of File Status		
14	1	End of Operation Status		
15	1	Unit Signal Status		
		INSTRUCTION EXCEPTION		
16	1	Operation Code Invalid	*	
17	1	Address Invalid	*	Execute, Load Indirect, Execute Indirect.
18	1	Incomplete Indirect Addressing	T	
19	1	EXECUTE Exception	T	Execute, Execute Indirect.
		ADDRESS MONITORING		
20	1	Data Write	*	
21	m	Data Read		
22	m	Instruction Read	E	
		OPERATION CODE MONITORING		
23	m	S only Operation Code	E	
24	m	H only Operation Code	E	
25	m	Successful Branch	T	Next Operation

* Execution of the instruction is stopped or the instruction is executed as a NO OPERATION.

E Interruption occurs before execution in systems for which operation code is meaningful. In other systems, operation is executed as a NO OPERATION.

	Mask			Reset By
		FLAGGING		
26	m	Index Flag	T	Next index refill operation
27	m	Data Flag T	T	Next operation which entails a data fetch
28	m	Data Flag U	T	
29	m	Data Flag V	T	
		RESULT EXCEPTION - GENERAL		
30	m	Lost Carry		
31	m	Incomplete Field		
32	m	Zero Divisor		
		RESULT EXCEPTION-FLOATING POINT ONLY		
33	m	Improper Floating Point Result		
34	m	Lost Significance		
35	m	Preparatory Shift > 48		
36	m	Exponent Overflow: Exp $\geq 2^{11}$		
37	m	$2^{10} \leq$ Exp $< 2^{11}$		
38	m	$2^{8} \leq$ Exp $< 2^{10}$		
39	m	$2^{5} \leq$ Exp $< 2^{8}$		
40	m	$-2^{11} <$ Exp $\leq -2^{10}$		
41	m	Exponent Underflow: Exp $\leq -2^{11}$		
42	m	Noisy Mode		
		STREAMING		
43	m	Memory Count Overflow		
44	m	Statistical Counter Overflow		
45	m	Statistical Accumulator Overflow, Underflow		
46	m	Statistical Accumulator Threshold		
47	m	End of Last Level P	T	Next use of P, Q, R, respectively.
48	m	End of Last Level Q	T	
49	m	End of Last Level R	T	
		This group of indicators only present in H System.		
		INDEX RESULT RANGE		
50	0	Index Count = 0	T	Next index modification.
51	0	Index Result < 0	T	
52	0	Index Result = 0	T	
53	0	Index Result > 0	T	
54	0	Index Comparison Result - Low	T	Next index comparison.
55	0	Index Comparison Result - Equal	T	
56	0	Index Comparison Result - High	T	

	Mask			Reset By
		ARITHMETIC RESULT RANGE		
57	0	Result < 0	T	Next non-comparison, arithmetic or connective operation.
58	0	Result = 0	T	
59	0	Result > 0	T	
60	0	Result Sign Minus	T	
61	0	Comparison Result - Low	T	Next comparison operation on accumulator.
62	0	Comparison Result - Equal	T	
63	0	Comparison Result - High	T	

Appendix 2: Floating Point

1. Divide

LASL asked if the unnormalized operation affected the speed of the normalized operation adversely. The reply was no. The timing penalty is on the unnormalized.

2. Rounding

LASL indicated that rounding was desirable for eliminating bias rather than to obtain an extra half bit of precision. The expected time penalties for rounding were stated by IBM as follows:

Add	20%
Multiply	10%
Divide	less than 10%

In view of these penalties it was agreed that the add and multiply operations would not be rounded (when rounding is required a double length result will be developed in the accumulator and the STORE ROUNDED operation will be used). DIVIDE (single) should be rounded if the time penalty does not exceed 10%. DIVIDE DOUBLE should not be rounded.

3. Noisy Mode

The present definition of this mode as it applies to add-type operations (that is, reversal of bit 47 of the result before normalization) fulfills the primary requirement of LASL (namely, detecting near-cancellation). No noise should appear when one floating point number with zero mantissa is subtracted from another floating point number with zero mantissa.

On non-add-type operations noise may be handled in whatever fashion proves to be most convenient from the design viewpoint (for example, in MPY the 47th bit may be reversed before normalization rather than after if this is more convenient).

4. Treatment of Zero

4.1 Consider two floating point numbers each having a zero mantissa. When one is added to or subtracted from the other, the result should have an exponent equal to the larger of the two operand exponents.

4.2 Whenever a single precision result is developed which before normalization has a zero mantissa, the left shift of 48 should be omitted and the exponent should not be reduced by 48. Similarly, whenever a double precision result is developed which before normalization has a zero mantissa, the left shift of 96 should be omitted and the exponent should not be reduced by 96.

Appendix 3: Interrupt System

3.1 LASL pointed out that one or more interrupts of the following kinds may occur when a branch instruction is executed:

	INTERRUPT	TYPE OF BRANCH
17	Address Invalid	Any
20	Data Write	Store IC, Branch on Bit
21	Data Read	Refill, Branch on Bit
22	Instruction Read	Any
25	Successful Branch	Any
26	Index Flag	Refill

In general it is necessary to know three things: the location of the branch instruction, the effective address to which branching should occur (conditionally or unconditionally) and whether or not the branch was successful. If a branch instruction generates one or more of the interrupts listed above and is allowed to change the contents of the instruction counter before the interrupt is taken, then information will be lost as to the location of the branch instruction.

It was further pointed out that, if execution of certain branch instructions, should not be completed when a synchronous interrupt is generated, then a consistent method of completing the secondary operations involved should be specified.

LASL proposed that whenever branching occurs the location of the branch instruction be unconditionally and automatically recorded in a register set aside for this purpose. IBM agreed that it was desirable to know the location of any instruction which generates an interrupt (at the time it is executed) and, further, agreed to seek a solution to this problem.

3.2 LASL expressed the desire to be able to make any entry in the interrupt table a NO OP instruction particularly in those positions for which the mask bit is always on. At present, this would result in certain instructions in the problem program not being completed.

3.3 LASL indicated that single instruction fix-ups should not drain the look-ahead. In their view, performance on certain problems would be seriously impaired if this did happen. It would be acceptable, if it were necessary to impose the restriction that the instruction make no reference to index registers (either for address modification or for index modification.).

3.4 When the Data Write interrupt is generated, the writing of data into memory is inhibited. LASL would prefer the writing operation to be completed. IBM pointed out that this would be incompatible with one of the purposes of this type of interrupt: namely, locking an area of memory.

3.5 Two points of information:

1) A successful branch to zero generates an invalid address interrupt on completion of the branching.

2) An unsuccessful branch to an invalid address does not turn on the invalid address indicator.

Appendix 4: Error Detection and Correction

LASL brought to IBM's attention the following extract from Article II Subsection 1 e Item (20):

> "During addition, subtraction, multiplication and division, the primary arithmetic unit will repeat its operation if an error is detected. If the fault is intermittent and does not occur the second time, the arithmetic unit will produce a correct result and continue on with the problem without further interruption. ."

IBM indicated that automatic repetition of an erroneous operation in the primary arithmetic unit would be included if it appreciably increased the mean error-free time for the system. At present, however, such a degree of checking in the arithmetic unit would be out of balance with that applied to the control section of the machine.

Appendix 5: Programming Systems

5.1 An outline was presented of IBM plans for an assembly program, simulator, supervisory program and formula translator. A programming language being explored by IBM was described in some detail. In this language a Fortran-like notation is used for operations instead of a more conventional mnemonic code. The symbolism for operands requires that field length, byte size, radix, etc., be included in the defining (rather than the operating) statements. The initial reaction of LASL was that a more elementary scheme was needed both as a stepping stone and as a regular tool.

5.2 A plea was made by LASL for a formula translator which would take care of the algebraic expressions only in a program, and leave the programmer free to specify the controlling logical operations in a language related closely to machine language.

5.3 LASL and IBM agreed that the number of special symbols available on a 407 represented a severe restriction on programming language. LASL notified IBM of its intention to go ahead and build a special typewriter with a large number of symbols, if in three month's time IBM was still unable to undertake the commitment.

5.4 A detailed account was given by IBM of objectives for an elementary supervisory program. LASL expressed approval of the objectives, but added that the memory space consumed by this program should not exceed 2000 words.

5.5 LASL agreed to draw up their requirements in the programming systems area and to clarify the extent of cooperation which they desire to see in the LASL and IBM efforts. This information will be forwarded by Mr. Voorhees to Mr. Heising before March 20.

Appendix 6: Console

6.1 LASL classified the uses which they plan for the console as follows:

1. Normal use-evolutionary change
   A problem has on-line output which allows the operator to decide on the next problem to be done. In this case one or more parameters can be set up in advance and introduced on a non-stop basis.

2. Normal use-revolutionary change
   The problem program is brought to a halt and changes are made which are not in accordance with a preconceived plan.

3. Program Debugging
   Total elapsed time for debugging a given program is of importance here. High priorities make on-machine debugging a necessity.

4. Machine Debugging
   This activity is the only one which requires controlling facilities tied directly to the CPU, Exchange, etc., independently of any program.

6.2 LASL proposed the following features:

1. A buffer on the typewriter to allow the operator to set up and verify the information he wishes to put into the machine.

2. A tape reader for paper tapes prepared remotely.

3. A set of 64 sense switches (each 3 way) and 64 lights tied by hardware to the switches on a one-to-one basis. These switches and lights can be set and read by the program or by the operator.

4. A further set of 64 lights controlled by the program only.

5. A facility to print out the contents of any memory location directly on a typewriter in hexadecimal. octal.

6.3 IBM described the operating station as currently defined:

1. Consists of three units each of which is optional: an inquiry station, a console, and a CRT display unit.

2. Can be operated by one or two persons.

3. Input equipment on the console:
3 switches: 8 gangs, 128 positions with light detent
32 binary toggle switches
25 columns of 10-key strip switch (probably 4 contacts on each)
7 columns of 10-key strip switch (probably 4 contacts on each)

4. Output equipment on the console:
32 binary lights (perhaps neons)
25 columns digital readout with at least 10 digits and blank, preferably 12 digits and blank
7 columns digital readout with 10 digits and blank, preferably 12 digits and blank
1 gong
1 Unit Signal button

5. Operation of console:
The signal button gives a unit signal to the Exchange in the usual way. There are no other control keys, and there is a single Reserved light which is turned on by the signal button and turned off when a READ instruction is received. A "click" is to be audible whenever it is turned off.

The console shares an adapter with an Inquiry Station (typewriter). The LOCATE instruction is used to select one or the other. Normally, the program always locates to the console, but upon receipt of certain information, the defining program locates to the typewriter for the next message.

The rotary switches are used as pseudo-potentiometers for entering data that is inherently analog. The binary toggle switches are used individually, some to define the uses of other parts of the console, and some to define conditions for problem programs (sense switches). The 25-column, strip switch is used for entering decimal or octal data or instructions. The 7-column switch is used for setting up a decimal or octal address. This may be a start or stop location for a program, or an address whose contents are to be displayed or changed.

The gong signals the system operator that his attention is required. The 32 binary lights are used individually to describe status conditions in the machine or the problem program. They may also be used to show the setting of the corresponding toggles. The 25-column digital display is used to display decimal or octal data or instructions. It may also be used to show the setting of the corresponding strip switch or the rotary switches. The 7-column digital display is used to display an address, or to show the contents of the corresponding strip switch or the rotary switches.

The Reserved light warns the user that his last console setting has not yet been read into the machine.

The console herein defined is to be packaged so that it can readily be manipulated by a seated user who can also see the CRT display, operate an IQS, and manipulate a small subset of the maintenance console without undue motion or inconvenience.

6.4 LASL indicated the need for a facility which would enable an operator to find out where and how the machine became trapped— when he has discovered that it is trapped. The activity initiated by the load button is to be modified so that it will be possible to preserve the contents of the instruction counter and accumulator.